

FACULDADE DE ENGENHARIA DA UNIVERSIDADE DO PORTO

Análise e Gestão de Horários II

Relatório Final do Projeto Integrador

**Alexandre Correia
João Duarte
João Ribeiro
Pedro Oliveira**



Licenciatura em Engenharia Informática e Computação

Orientador: Daniel Castro Silva

1 de julho de 2024

Resumo

Este projeto teve como principal objetivo a otimização de um sistema de gestão de horários inicialmente desenvolvido por um grupo de estudantes do ano letivo anterior. A nossa equipa concentrou-se em aprimorar um sistema pré-existente utilizado pela comissão de horários do Departamento de Engenharia Informática (DEI), visando proporcionar mais eficiência e simplicidade no processo de edição e alteração de horários que ocorre semestralmente.

Os nossos esforços resultaram em melhorias significativas nos seguintes aspectos:

- *Tempos de parse*: O processo de *parse* envolve a leitura e interpretação de dados de horários em formato HTML para serem utilizados no sistema. A eficiência deste processo foi melhorada, reduzindo o tempo necessário para converter e carregar os dados em 62%.
- *Exportação de alterações*: a exportação de alterações feitas nos horários foi facilitada, permitindo que as modificações sejam facilmente compartilhadas e integradas em outros sistemas ou documentos. Esta funcionalidade é crucial para a comunicação e implementação de mudanças de forma eficaz.
- *Interface gráfica*: A interface gráfica do utilizador (GUI) foi aprimorada através do ajuste de alguns detalhes que tornam o sistema mais intuitivo de utilizar.
- *Base de dados*: Houve uma otimização no dimensionamento da base de dados, com uma redução de 40%, a par de uma incorporação de tabelas com mais informação útil.

Estas alterações permitirão uma melhor gestão de recursos computacionais e que se traduzirá numa melhor experiência de utilizador no futuro.

Conteúdo

1	Introdução	1
1.1	Enquadramento	1
1.2	Objetivos e resultados alcançados	2
1.3	Estrutura do relatório	2
2	Metodologia utilizada e principais atividades desenvolvidas	3
2.1	Metodologia utilizada e recursos	3
2.2	Intervenientes, papéis, responsabilidades e distribuição de notas	4
3	Desenvolvimento da solução	6
3.1	Arquitetura Existente	6
3.2	Requisitos	7
3.3	Tecnologias	8
3.4	Plano de trabalhos	8
3.5	Solução desenvolvida	9
3.5.1	Parse	9
3.5.2	Alterações à base de dados	9
3.5.3	Interface	10
3.5.4	Exportação das alterações	13
4	Conclusões	16
4.1	Resultados alcançados	16
4.2	Lições aprendidas	16
4.3	Sugestões futuras	17
	Referências	18

Capítulo 1

Introdução

A geração de horários é um problema NP-Complete (non-deterministic polynomial time) que muitas organizações e instituições enfrentam, principalmente na área do ensino [1]. Descobrir uma configuração eficaz que satisfaça as necessidades e restrições de todos os envolvidos (maioritariamente alunos e professores) é, portanto, uma tarefa difícil. Adicionalmente, é necessário conjugar as disponibilidades da faculdade no que diz respeito a salas e horários de docentes. É também necessário equilibrar a carga horária dos alunos e evitar sobrecargas de disciplinas, garantindo que os períodos de aula sejam distribuídos adequadamente ao longo da semana.

1.1 Enquadramento

Na Faculdade de Engenharia da Universidade do Porto (FEUP), a geração semestral de horários para docentes e alunos é um procedimento crucial, porém notoriamente complexo e moroso. A complexidade deste procedimento deve-se à necessidade de conciliar diversas restrições e preferências. É necessário equilibrar as disponibilidades de professores e alunos, garantir a utilização eficiente das salas, evitar conflitos de horários entre unidades curriculares e distribuir a carga horária de forma equitativa ao longo da semana. Adicionalmente, cada semestre pode introduzir novas variáveis e ajustes que complicam ainda mais o processo. Apesar destas dificuldades, há uma tendência na direção da automatização deste processo [2].

O processo atual de geração dos horários na FEUP envolve várias etapas principais:

- Geração de horários iniciais, usando um software especializado, tendo em conta restrições de docentes e salas;
- Edição destes horários por parte das comissões de horários de cada curso, que tenta verificar se todas as restrições são cumpridas, otimizar a utilização das salas, e ainda agrupar as aulas dos docentes, melhorando os seus horários individuais. Esta etapa pode ser repetida até que os horários do curso se considerem otimizados;
- Envio das alterações para a equipa de gestão de sistemas de informação, que as aplica aos horários iniciais, criando os horários oficiais.

A elaboração atual, ainda que "semi-automática", de horários académicos é uma tarefa suscetível a erros e que consome um tempo considerável dos docentes responsáveis pelo processo [3]. Ao mesmo tempo, os resultados muitas vezes não são os mais otimizados, ocasionando problemas na distribuição equitativa das atribuições para os docentes e dificuldades em ajustar a carga horária de forma eficiente para os alunos.

Perante esta realidade, surge a necessidade de desenvolver soluções que simplifiquem e aprimorem o processo de geração de horários na FEUP. Este trabalho em particular debruça-se sobre a melhoria da segunda etapa, procurando facilitar a tarefa de edição dos horários.

1.2 Objetivos e resultados alcançados

Neste contexto, este trabalho propõe-se a aprimorar e utilizar uma plataforma que permite a edição de horários cujo desenvolvimento já foi iniciado, mas que se encontrava incompleta [4]. No entanto, resolvendo alguns problemas cruciais, e com a introdução de mais funcionalidades adequadas, tornar-se-á numa ferramenta útil na edição de horários personalizados. Ao simplificar o processo de edição de horários, espera-se alcançar benefícios tangíveis principalmente em termos de economia de tempo, redução de erros e melhor utilização dos recursos disponíveis.

Por meio da implementação bem-sucedida dessa solução, espera-se estabelecer um novo padrão de excelência na gestão académica da FEUP, contribuindo para a eficiência operacional e para uma experiência mais satisfatória tanto para os docentes quanto para os alunos.

Inicialmente, eram esperados resultados concretos, como uma redução efetiva no tempo médio gasto pelos docentes encarregues desta tarefa de elaboração de horários através da implementação de algoritmos otimizados e uma reformulação da interface já criada pelos estudantes do ano lectivo anterior, de forma a alcançar uma experiência de utilizador renovada.

Os resultados alcançados, em suma, foram de encontro aos objetivos estabelecidos, na medida em que foram conseguidas excelentes melhorias no tempo do *parser*, assim como na formulação da base de dados que se traduziram em melhorias de desempenho do sistema. A par destas modificações, foi também realizada uma reformulação na forma como as alterações finais são exportadas, o que permitirá e facilitará um eventual melhoramento em trabalhos futuros neste tema. A nível da interface gráfica procurou-se aprimorar a estrutura existente, garantindo que o utilizador tem ao dispor toda a informação de que necessita para realizar as alterações adequadas ao horário.

1.3 Estrutura do relatório

Para além desta parte introdutória, este relatório está dividido em três secções. Sendo elas:

1. **Metodologia:** Nesta secção é descrita detalhadamente a metodologia utilizada no desenvolvimento do projeto. Esta parte inclui a identificação dos autores e as suas responsabilidades, bem como os stakeholders envolvidos.
2. **Desenvolvimento da Solução:** Aqui é abordado o desenvolvimento da solução, descrevendo os resultados esperados, os requisitos e as restrições associadas. Além disso, é apresentada a solução desenvolvida e os métodos de validação aplicados.
3. **Conclusão:** Na secção final, são resumidos os resultados alcançados e oferecidas sugestões para eventuais trabalhos futuros.

Capítulo 2

Metodologia utilizada e principais atividades desenvolvidas

Neste capítulo é descrita a metodologia seguida, são enumerados os principais intervenientes no projeto e é apresentado o registo das principais atividades desenvolvidas.

2.1 Metodologia utilizada e recursos

Para o desenvolvimento deste projeto, foi seguido um processo de desenvolvimento *Agile* de software, similar ao *Scrum*. Esta abordagem iterativa com feedback frequente do professor Daniel Castro Silva serviu para garantir uma adaptação constante às necessidades do projeto e a melhoria contínua das soluções desenvolvidas. A metodologia utilizada incluiu:

1 - Desenvolvimento iterativo:

- Foi adotado um modelo de desenvolvimento iterativo, dividindo o projeto em etapas menores e trabalhando em ciclos de iteração.
- Em algumas situações, foi necessário inverter este modelo e optar por um foco no desenvolvimento de tarefas mais complexas, como o caso do *parser*, e do módulo de exportação de alterações.

2 - Reuniões Semanais de Acompanhamento:

- Foram realizadas reuniões semanais com o docente para apresentar o progresso, discutir desafios encontrados e receber orientações.
- Essas reuniões foram fundamentais para garantir alinhamento com as expectativas do projeto e para receber feedback regularmente.
- Em semanas mais sobrecarregadas com exames, estas reuniões não se realizaram, pois por vezes o progresso no projeto não diferia substancialmente da semana anterior.

3 - Comunicação Frequente entre os Membros da Equipa:

- Além das reuniões com o docente, foi mantida uma comunicação frequente entre os membros da equipa.
- Isto permitiu uma melhor eficácia na resolução de problemas, e alinhamento na distribuição das tarefas ao longo do projeto.

No que diz respeito aos recursos utilizados durante o desenvolvimento do projeto, e tendo em conta que era imprescindível haver uma comunicação constante não só entre os membros da equipa, mas também com o professor Daniel Castro Silva, foram utilizados os seguintes recursos:

- **GitHub (Controlo de Versão):**
 - O código-fonte do projeto foi armazenado e gerenciado utilizando o Git e o GitHub.
 - Foram utilizadas ferramentas do GitHub, como *pull requests* e *issues*, para revisões de código e rastreamento de problemas.
- **Ferramentas de Desenvolvimento:** Para codificação e desenvolvimento, foram utilizados IDEs (Ambiente de Desenvolvimento Integrado) como IntelliJ IDEA, Eclipse ou Visual Studio Code, sendo esta uma questão de decisão pessoal.
- **Recursos de Comunicação Online:** Para facilitar a colaboração remota, foram empregues ferramentas de comunicação online, sendo elas o *Zoom* nas reuniões iniciais com o professor Daniel, passando posteriormente para a plataforma *Discord* para chamadas entre o grupo, e mesmo para algumas reuniões semanais com o docente.
- **Elaboração do relatório:** Para a criação e elaboração deste relatório, foi utilizada a plataforma *Overleaf*. Esta ferramenta permitiu que todos os membros da equipa colaborassem simultaneamente e em tempo real, facilitando a integração de contribuições e a revisão contínua do documento. Como resultado, foi alcançada uma maior consistência e precisão nas informações apresentadas.

2.2 Intervenientes, papéis, responsabilidades e distribuição de notas

Esta secção descreve os intervenientes externos ao nosso projeto, bem como a distribuição de tarefas e a complementaridade com a tese de Mestrado do aluno Miguel Lopes.

- **Miguel Lopes:**
 - **Papel:** Interveniente e colaborador no contexto da sua tese de mestrado.
 - **Responsabilidades:** Contribuir com conhecimento específico e orientações relacionadas ao tema da tese, interagindo com a equipa para alinhar o projeto com os objetivos académicos.
- **Comissão de Horários do DEI:**
 - **Papel:** Stakeholder institucional responsável pela gestão e organização dos horários académicos no DEI.
 - **Interação:** Possível interação para discussões sobre requisitos e considerações específicas relacionadas ao contexto académico e horários de aulas.

Relativamente à distribuição de tarefas ao longo do desenvolvimento do projeto, e devido à dificuldade e complexidade do mesmo, concluiu-se que a divisão de tarefas foi crucial para o bom desenvolvimento geral do projeto. Inicialmente, todos participaram numa das principais tarefas que consistiu na melhoria do *parser* a nível de performance temporal. Posteriormente, as tarefas foram divididas de forma igualitária e colaborativa levando em consideração as habilidades individuais, interesses e disponibilidade de cada membro. As responsabilidades foram atribuídas com base nessas considerações com o objetivo de otimizar a eficiência e qualidade do trabalho realizado. À medida que o projeto foi avançando, algumas tarefas foram distribuídas individualmente ou mesmo a pares, de forma a garantir uma melhor organização e progresso no projeto. Na reta final de desenvolvimento existiram mais reuniões e colaboração entre

todos os membros, uma vez que existiam alguns pontos a melhorar para a apresentação final, onde sublinhamos e salientamos a importância do tutor Daniel Castro Silva, crucial no esclarecimento de dúvidas.

A par do nosso trabalho, o aluno de Mestrado do M.EIC Miguel Lopes, também sob a orientação do Professor Daniel Silva, procurou desenvolver uma plataforma de otimização automática de horários, com base em Inteligência Artificial, que usa a base de dados produzida pela nossa aplicação. As alterações à aplicação foram concebidas considerando também a complementaridade com o projeto da tese de Mestrado do Miguel. Ele esteve presente ao longo do desenvolvimento da nossa aplicação, e a equipa procurou alinhar os objetivos e resultados com os interesses e necessidades académicas identificadas na tese.

Distribuição de notas

- João Ribeiro: 25%
- Alexandre Correia: 25%
- João Duarte: 25%
- Pedro Oliveira: 25%

Capítulo 3

Desenvolvimento da solução

Neste capítulo é descrito o trabalho desenvolvido com o objetivo de alcançar os resultados esperados. É feita inicialmente uma pequena introdução relativamente à estrutura e arquitetura do projeto que foi desenvolvido no ano letivo anterior e são também abordados os requisitos essenciais para o desenvolvimento da solução.

3.1 Arquitetura Existente

Ainda antes de iniciar este projeto, a aplicação apresentava uma arquitetura sólida, e bem definida (Figura 3.1). Identificaram-se vários módulos principais, e sobre estes foram realizadas alterações e melhorias. No funcionamento normal da aplicação, os horários gerados automaticamente, em formato HTML, são lidos para uma base de dados SQL usando o módulo de *parse*. No final deste processo, a base de dados é duplicada, para permitir à aplicação determinar quais as alterações que foram realizadas pelo utilizador.

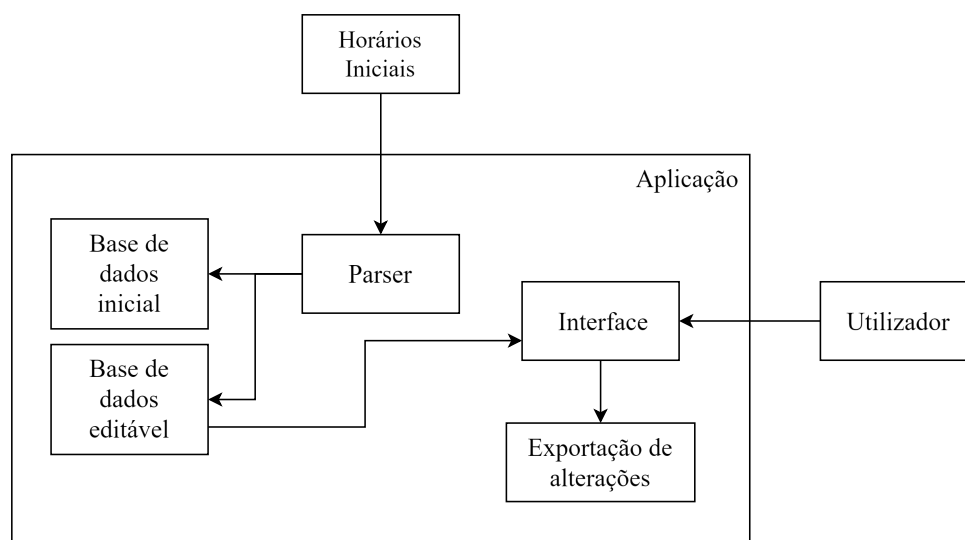


Figura 3.1: Arquitetura da aplicação existente

Após o preenchimento da base de dados, o utilizador pode visualizar os horários lidos usando a interface gráfica da aplicação. Esta permite selecionar o curso, o ano, o turno, a turma, e o período de tempo dentro do semestre a visualizar (caso se aplique). Nesta interface gráfica, é permitido ao utilizador realizar trocas nos horários. Estas podem ser feitas diretamente, ou seja, clicando sequencialmente nos dois atributos a trocar em duas aulas diferentes - o docente, a sala, ou mesmo a aula completa, que pode ser movida para outro lugar do horário - ou usando o menu lateral que permite uma edição mais abrangente, incluindo

adicionar ou remover docentes e salas. Tendo realizado todas as alterações desejadas, o utilizador pode usar o módulo de exportação de alterações para exportar, num formato de linguagem natural sintática, uma lista com estas alterações. Alguns destes elementos e processos apresentavam erros e/ou problemas de performance. Foram estas falhas que a equipa se propôs a colmatar, assim como acrescentar mais funcionalidades úteis ao sistema de edição de horários.

3.2 Requisitos

Os principais requisitos do trabalho passaram por melhorar e adicionar funcionalidades à aplicação web já existente. Para dividir o trabalho de forma funcional, consideraram-se dois sistemas principais na aplicação: o *parser* das páginas HTML, e a interface gráfica. No lado do *parser* de horários HTML:

- Garantia de correção da informação recolhida: Foi assegurado que os dados extraídos das páginas HTML são precisos e correspondem exatamente às informações exibidas. Isto envolveu a validação e verificação dos dados recolhidos para garantir integridade.
- Melhorias de desempenho: Estas otimizações foram obtidas ao reduzir o tempo necessário para processar e extrair informações das páginas HTML.
- Completude da informação: Foi garantido que todas as informações relevantes foram capturadas e processadas pelo *parser*. Isto significa que não apenas os dados básicos como horários e salas foram extraídos, mas também dados adicionais, tais como detalhes sobre docentes, turmas, e unidades curriculares.

No lado da interface gráfica:

- Melhorias de interface e interação
 - Correção de problemas na visualização dos horários;
 - Adição de funcionalidade de visualização de horários antes e após alterações;
 - Permitir visualização do horário por UC;
 - Permitir visualização de horário por docente;
 - Em UCs que funcionam por "blocos" de semanas (e.g. Comunicação Profissional), na visualização por UC conseguir ver esses blocos separadamente.
- Trocas de aulas
 - Ao tentar trocar as salas, quando é clicada uma sala, avaliar a disponibilidade de salas. Marcar de alguma forma as salas para onde a aula não pode ser movida;
 - Garantir que as verificações da possibilidade de trocas têm em conta as semanas em que as aulas decorrem;
 - Ao realizar trocas de docentes ou salas, ter uma lista com os 'sugeridos'. Por exemplo, docentes da mesma UC, salas do mesmo tipo com capacidade semelhante.
- Outros
 - Ter em conta as semanas das aulas também nos horários dos docentes. Adicionar aviso sobre potenciais mudanças de horário durante o semestre.

Um último requisito passava pela melhoria visual e de mais fácil interpretação da funcionalidade de exportação de alterações realizadas aos horários, num formato conciso e de simples interpretação.

3.3 Tecnologias

A aplicação web encontra-se implementada em Python, tanto do lado do *backend*, usando Django, e do lado do *frontend* era usado JavaScript simples, com *Bootstrap* para facilitar a criação de alguns componentes. No *backend*, a estrutura já existente era sólida, sendo apenas necessárias alterações ao módulo de *parse*. Para a tecnologia de *frontend* foram consideradas outras soluções:

- FullCalendar [5] - um calendário em JavaScript, orientado a React, mas com suporte para Vue e Angular. Esta opção apresentava versatilidade na personalização do calendário, suporte para *drag & drop* nas trocas, e bom apoio ao desenvolvimento, com uma comunidade ativa. No entanto, a curva de aprendizagem aparentava ser bastante íngreme, e a edição de eventos era limitada, o que não iria de encontro às nossas necessidades.
- ToastUI Calendar [6] - também um calendário JavaScript, open-source, com suporte para React e Vue. Esta opção oferecia documentação bastante completa, funcionalidades para edição de eventos, e ainda *drag & drop* de eventos. A principal desvantagem era a falta de suporte, dado que não foi encontrada nenhuma versão posterior a 2022.
- React-big-calendar [7] - um componente React, inspirado no FullCalendar. Aparentava ser de fácil utilização, e é personalizável com Sass [8]. Esta opção trazia também uma funcionalidade *drag & drop*, com bom suporte comunitário (*open-source*). A principal desvantagem era a necessidade maior de personalização, dado que o aspeto "out of the box" não era tão apelativo como as outras opções.

No final, tendo em conta estas opções, que pareceram ser as mais adequadas às necessidades do projeto, optou-se por manter a *stack* de tecnologias *frontend* já existente. Isto deveu-se principalmente a atrasos nas fases iniciais de análise de código e melhoria do *parser*, que resultaram num prazo mais limitado para a melhoria da interface gráfica. A decisão do grupo passou então por manter a tecnologia existente para o *frontend*, e tentar compreender e melhorar as funções e componentes com falhas e erros.

Relativamente à arquitetura, de um modo global foi mantida a maior parte da estrutura, sendo que as alterações principais foram realizadas a componentes individuais da aplicação.

3.4 Plano de trabalhos

Nesta secção é apresentado um cronograma temporal detalhado das diferentes fases do projeto. Este foi elaborado de forma a podermos garantir um planeamento mais eficaz, e uma execução e desenvolvimento mais organizado em todas as etapas do projeto, conforme verificado na Fig. 3.2.

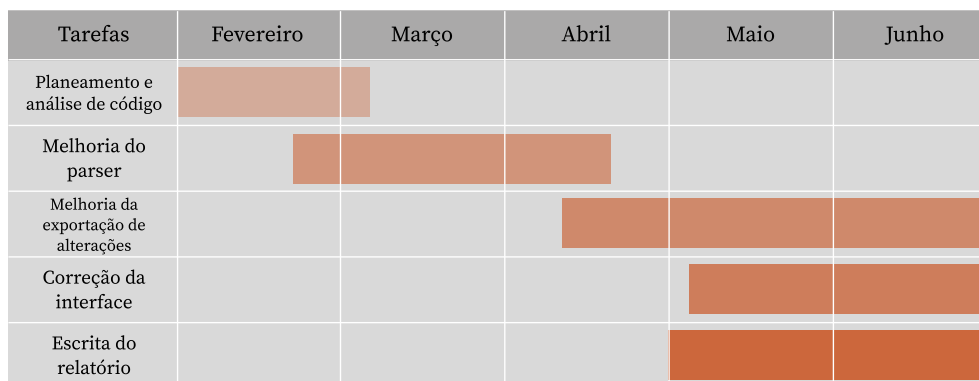


Figura 3.2: Diagrama de Gantt com as diferentes fases do projeto

Este cronograma final sofreu alterações relativamente ao planeamento inicial, principalmente na etapa de melhoria do *parser*, que se esperava ter sido resolvida num espaço de tempo mais curto. No entanto, foi necessário alargar este prazo, assim como o da melhoria do módulo de exportação de alterações, que requeria redefinir e redesenhar a forma como estas eram apresentadas em linguagem natural ao utilizador.

3.5 Solução desenvolvida

Nesta secção abordamos a solução que foi desenvolvida ao longo do projeto através duma descrição dos pontos que se consideram ser as principais soluções deste projeto.

3.5.1 Parse

A página onde se encontram os horários iniciais em HTML - gerados automaticamente - está dividida em quatro secções principais - docentes, turmas, salas, e unidades curriculares. Na versão inicial da aplicação, o *parser* realizava uma leitura das secções dos docentes, turmas e unidades curriculares, de modo a recolher todos os dados essenciais para a base de dados. Em cada página de horário de cada uma destas secções, existem também blocos vermelhos, que indicam zonas nos horários de docentes, salas e turmas onde estes se encontram ocupados por algum motivo, e por isso estes blocos estão impedidos de receber outras aulas. Também na versão inicial, era recolhida e armazenada a informação relativa a estes blocos. Porém, este processo era realizado de uma forma ineficiente, envolvendo uma repetição desnecessária de linhas na tabela de blocos vermelhos na base de dados, o que, não só ocupava bastante tempo no processo de *parse* dos horários, como também aumentava a dimensão da base de dados.

A nossa análise ao módulo de *parse* existente e às páginas dos horários iniciais revelou que a leitura da secção de unidades curriculares era redundante, e que toda a informação recolhida neste passo podia ser recolhida na leitura da secção de turmas. Para além disto, a secção de unidades curriculares não se encontrava completa. Em particular, nos casos de unidades curriculares que ocorrem em simultâneo para dois cursos diferentes, mas com diferentes nomes em cada curso, apenas uma das unidades curriculares era listada. Já nas páginas de horários de turmas, ambas as unidades curriculares são apresentadas, e é possível recolher a informação relativa a todas as unidades curriculares, sem falta. Por estes motivos, as funções associadas à leitura das páginas de horários de unidades curriculares foram removidas, e as funções associadas à leitura das páginas de horários de turmas foram adaptadas para recolher mais informação.

Durante este processo de alteração, foram também identificados vários locais no código onde a performance era mais reduzida, principalmente sob a forma de ciclos com iterações a mais, ou até ciclos desnecessários. Estes foram também corrigidos e/ou removidos, de modo a eliminar o máximo de operações redundantes ou dispensáveis.

Em relação aos resultados obtidos, foi possível, como refletido na Figura 3.3, uma redução da dimensão da base de dados assim como do tempo de *parse* em 40% e 62% respetivamente, resultando numa melhor performance do sistema.

3.5.2 Alterações à base de dados

Inicialmente, devido à forma como o sistema estava implementado, verificou-se que os blocos vermelhos (entidades referentes a uma indisponibilidade/conflito no horário) estavam a ser lidos repetidamente, o que acabava por afetar a performance do sistema e atrasar a interação entre o sistema e o utilizador. Ao encontrar blocos vermelhos no processo de *parse*, estes eram guardados numa tabela que armazenava apenas o dia e hora de cada bloco. Estas entradas eram imediatamente usadas como referência para

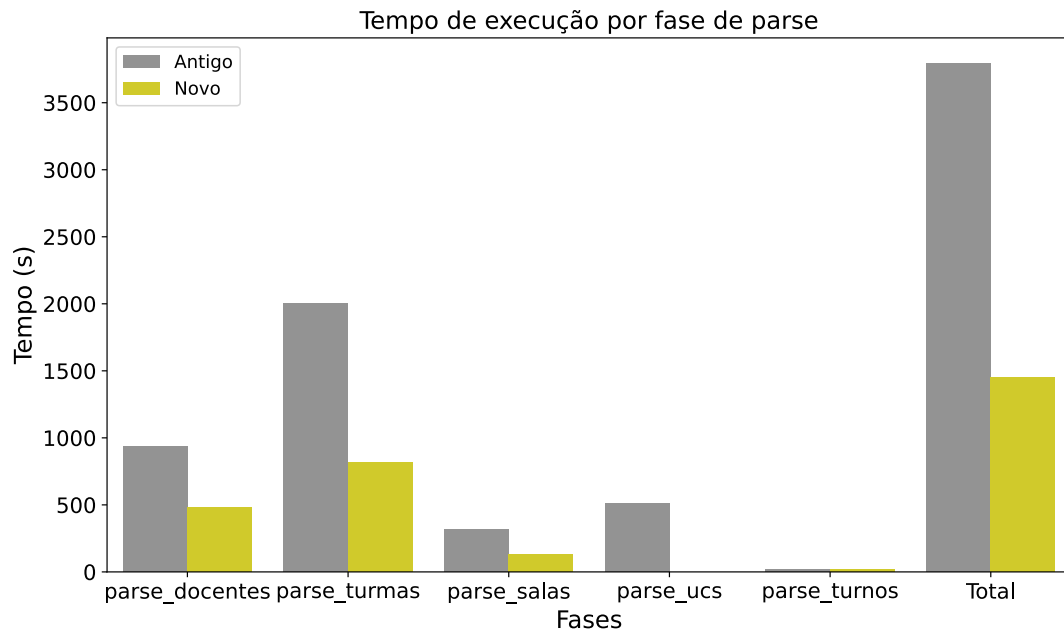


Figura 3.3: Comparação do tempo das diferentes etapas de *parse* entre a versão anterior da aplicação e a existente

preencher as tabelas de associação de blocos vermelhos salas ou docentes. Dado que o número de blocos vermelhos possíveis no horário está limitado pelo número de entradas/*slots* desse horário, existia um grande número de entradas repetidas na tabela de referência, e uma porção significativa de tempo de *parse* era desperdiçada nesta tarefa.

A principal alteração neste mecanismo envolveu o preenchimento automático inicial de uma tabela de tamanho fixo que serve como referência para todos os blocos vermelhos possíveis num horário. Durante o processo de *parse*, quando são encontrados blocos vermelhos, a sua informação é colocada em tabelas que os associa a salas ou docentes, fazendo referência à tabela de tamanho fixo.

Assim, a forma como os blocos vermelhos são lidos foi reestruturada e foi encontrada uma solução que reduziu drasticamente o número de blocos lidos desnecessariamente. No final, foi possível reduzir a quantidade de blocos vermelhos que eram gerados na tabela de referência de 98.808 blocos para 180 blocos. Esta redução traduziu-se numa melhoria significativa na performance do sistema, dado que desta forma não são desperdiçados recursos computacionais em operações redundantes e desnecessárias.

Também na base de dados, passou a ser possível armazenar informação relativa a aulas simultâneas. No contexto dos horários da FEUP, e desta aplicação em particular, aulas simultâneas são aquelas que são assistidas por cursos diferentes, com nomes diferentes em cada curso, mas que são dadas pelo mesmo docente, na mesma sala, no mesmo dia e hora. Armazenar informação sobre estas aulas é particularmente relevante já que, dadas as restrições físicas e temporais que as agrupam, quando uma delas sofre alterações de sala ou de dia/hora, a outra deverá sofrer simultaneamente as mesmas alterações. Para que isto fosse possível, foi necessário adicionar tabelas à base de dados, e um passo no processo de *parse* para encontrar estes casos, após todas as aulas serem lidas.

3.5.3 Interface

A interface da versão inicial e da versão melhorada permite seleccionar, dentro de um dado curso e ano, um turno e/ou uma turma para visualizar de forma individual. No entanto, esta funcionalidade não se encontrava completamente operacional, principalmente na seleção dos turnos. Por exemplo, no curso

L.EIC, ano 1, após selecionar um turno, não era possível observar corretamente o horário completo deste, e as páginas incluíam sempre artefactos como células da tabela em falta, ou colunas de turmas repetidas várias vezes no mesmo dia (Figura 3.4).

[illegible]

Figura 3.4: Identificadores repetidos, células a mais, e células em falta na aplicação inicial

[illegible]

Figura 3.5: Identificadores corrigidos, células deixam de estar a mais ou em falta

A abordagem a esta questão envolveu a alteração da lógica por trás desta funcionalidade, principalmente pela inclusão de mais informação nos elementos HTML da tabela, o que facilita o mecanismo de mostrar/ocultar estes elementos usando JavaScript aquando da seleção dos turnos. Ao alterar a lógica existente, foi possível deixar de observar a existência de artefactos, e passar a conseguir ler corretamente os horários de cada turno (Figura 3.5). Para além disto, foram também encontrados casos de cursos, em particular o 1º ano da L.EIC, onde uma ou mais unidades curriculares têm um número diferente de turnos das restantes. Nestes casos, passamos a apresentar no *dropdown* de seleção dos turnos opções que mostram o conjunto de turmas associado aos turnos destas unidades curriculares, que serão, naturalmente, diferentes dos restantes turnos desse ano. No exemplo concreto do 1º ano da L.EIC (horário do 2º semestre), o menu de seleção de turno passa a apresentar os turnos 1, 2 e 3, e ainda AMvII 1 e AMvII 2, apresentados na Figura 3.6, sendo que estes últimos correspondem aos dois turnos de Análise Matemática II, cuja distribuição de turmas por turnos é diferente. Assim, para qualquer seleção de turno, é mostrado o horário completo de todas as turmas desse turno.

Ainda na seleção da informação a visualizar, foi melhorada a seleção das semanas do horário. A lógica principal desta funcionalidade foi mantida, continuando a envolver um pedido ao servidor pelas aulas de

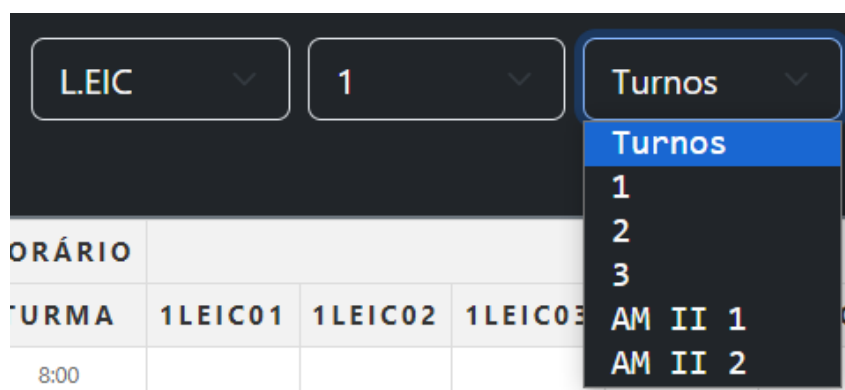


Figura 3.6: Opções de turnos para o 1º ano da L.EIC

um dado curso, sendo que este apenas devolve as que correspondem ao período pretendido. A abordagem passou essencialmente pela melhoria do processo de filtragem das aulas a devolver ao cliente. Durante a correção desta funcionalidade, foi também conseguida uma melhoria do processo inicial de seleção de curso e ano, que envolvia dois pedidos mais "pesados" ao servidor, um após cada seleção. Estes pedidos foram reduzidos para apenas um, que ocorre após selecionar o curso e o ano, o que oferece um menor tempo de espera ao utilizador.

Relativamente aos blocos de aulas no horário, foi também realizada uma alteração na visualização destes, a par com a ordenação correta das turmas nas colunas do cimo do horário. Na versão inicial da aplicação, quando não está selecionado um turno em particular, as turmas estão ordenadas pelo turno a que pertencem e não pela sua ordem numérica (p. ex.: 2LEIC01, 2LEIC02, 2LEIC03, etc.). Ao corrigir esta ordenação, dada a composição de alguns turnos, foi necessário separar em vários blocos as aulas que pertenciam a turmas não consecutivas na sua ordenação. Mais uma vez, no caso concreto do 1º ano da L.EIC, o turno 1 é composto pelas turmas 1LEIC01, 1LEIC02, 1LEIC12 e 1LEIC13. Dado que turmas nestas condições, na nossa versão, deixam de ser contíguas na tabela, foi necessário dividir aulas dadas a todas as turmas de turnos assim em vários blocos, de modo a visualizá-las corretamente (Figura 3.7).

SEGUNDA													
1LEIC01	1LEIC02	1LEIC03	1LEIC04	1LEIC05	1LEIC06	1LEIC07	1LEIC08	1LEIC09	1LEIC10	1LEIC11	1LEIC12	1LEIC13	1LEIC14
F I CDP FJBS B007	P DCC - ERBM PNFRCD B001										F I CDP FJBS B007	F I PTPV B219	
F I PTPV B219	AC LEIC_ACDoc4 B110	AM II AMAN B318	F I CDP B222	AM II IARL B322	F I DCC - JB B331	P DCC - SPRAR B207	TC JAC B119	P DCC - ACM B301	AC AJA B101	AC IAFVP B314	AC JCF B103	F I ATC B330	

Figura 3.7: Aula separada em células diferentes para correspondência com as turmas a que pertence

Para facilitar o processo de trocas de docentes e salas, foram adicionados indicadores visuais para mostrar ao utilizador os blocos onde um dado docente ou sala já estão ocupados. Estes são determinados tendo em conta todas as aulas lidas na base de dados, e são independentes dos blocos vermelhos que já existem nos horários iniciais, mencionados na secção anterior. Ao selecionar uma célula de uma aula, esta funcionalidade pinta as células onde o docente está ocupado e também as células onde a sala está a ser utilizada (Figura 3.8). É usada uma terceira cor para sinalizar células onde tanto o docente como a sala não têm disponibilidade. Caso sejam selecionados o docente ou a sala, são apenas pintadas as células correspondentes a um ou ao outro.

Com o objetivo de validar estas alterações e correções realizadas à interface, realizámos apenas uma análise visual da informação. Neste sentido, eram observados os horários iniciais em HTML e comparados

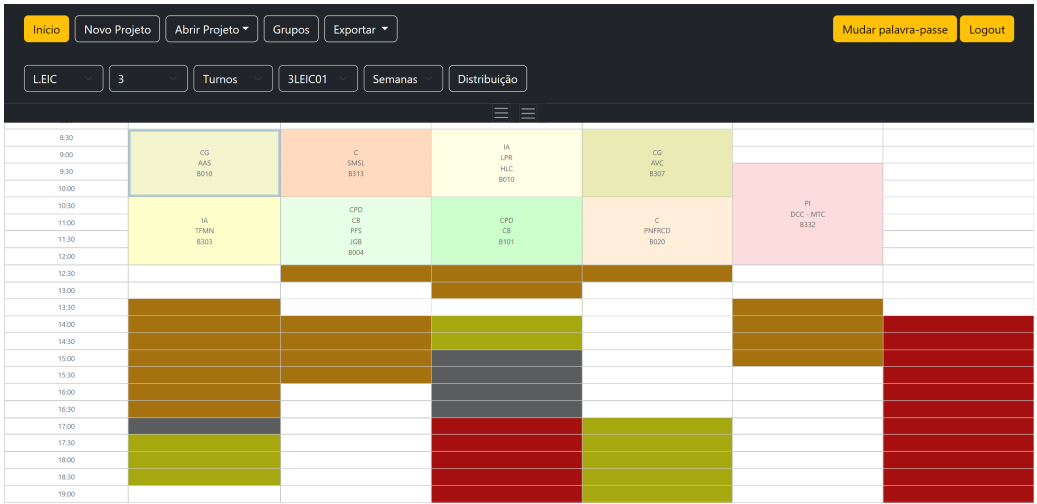


Figura 3.8: Identificação de células onde sala e/ou docente da aula não têm disponibilidade

com os horários apresentados na aplicação, de modo a verificar se toda a informação era apresentada corretamente. Ao mesmo tempo, era também executada, sempre que necessário, uma versão inicial da aplicação, procurando perceber quais os locais com erros que ainda persistiam, e quais já se encontravam resolvidos.

3.5.4 Exportação das alterações

A exportação original da aplicação web é gerada após uma comparação entre a base de dados atualizada com as alterações feitas pelo utilizador nos horários e a base de dados inicial, anterior a qualquer modificação. Através desta comparação, são identificadas as alterações realizadas em cada tabela da base de dados, que, após serem processadas adequadamente, são transformadas numa mensagem de texto.

Esta abordagem inicial apresentava alguns problemas relacionados com a estrutura da mensagem de texto da exportação, que não era suficientemente concisa e simples de interpretar pela equipa de gestão dos sistemas de informação, responsáveis por efetuar essas alterações no Sigarra. Além disso, essa entidade reportou um problema recorrente durante as alterações sequenciais (manuais) realizadas na plataforma do Sigarra. Este problema relacionava-se com a ocorrência de conflitos nos horários existentes, devido à indisponibilidade de salas, docentes ou turmas. Assim, durante a aplicação dessas alterações, era crucial identificar onde esses conflitos ocorriam e como eram resolvidos.

Nó	Tabela da BD	Alteração
1	aula	Aula 774 é alterada de Sexta, 10h30 para Quinta, 8h30
2	aula	Aula 853 é alterada de Quinta, 8h30 para Sexta, 10h30
3	aula	Aula 771 é alterada de Terça, 8h30 para Segunda, 16h00
4	aulaSala	Aula 853 troca da sala B301 para B336
5	aulaSala	Aula 774 troca da sala B336 para B301
6	aulaSala	Aula 771 troca da sala B334 para J002
7	aulaDocente	Aula 853 troca do docente 231072 para 522405
8	aulaDocente	Aula 774 troca do docente 522405 para 231072

Tabela 3.1: Alterações na base de dados

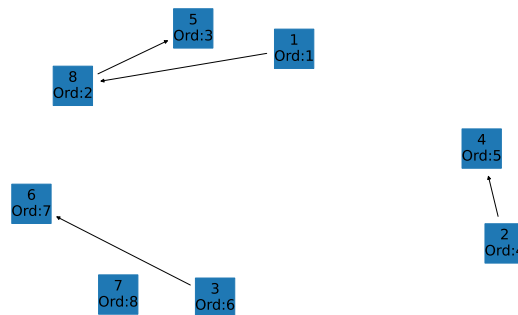


Figura 3.9: Grafo de exportação

A solução encontrada para este problema envolveu a criação de um grafo direcionado, cujos nós representam uma alteração isolada num horário, como por exemplo a mudança de sala de uma aula. As arestas ligam dois nós, onde o nó de origem da aresta representa uma alteração que causa um conflito na base de dados, e o nó de destino da aresta representa a alteração necessária para resolver o conflito anterior.

A Tabela 3.1 apresenta um conjunto exemplo de alterações realizadas na turma 1LEIC01, do segundo semestre do ano letivo 2023/2024. Cada um dos nós da tabela está representado no grafo de exportação da Figura 3.9, com as respetivas arestas que refletem a relação de conflito entre alterações.

É possível observar no grafo de exportação que existem alguns conjuntos menores de alterações que estão interligados (1-8-5, 2-4 e 3-6). Isto significa que há conflitos que são gerados e resolvidos nessa sequência de alterações. Deste modo, na sequência 1-8-5 observa-se que estão a ser realizadas alterações na aula com id 774, onde após a alteração para um dia de semana e uma hora diferente ocorrem conflitos de disponibilidade de sala e docente, que são resolvidos nas alterações 8 e 5. O mesmo acontece de forma semelhante nas restantes sequências. A ordem de leitura deste grafo para construir a mensagem de exportação em linguagem natural é identificada pelo parâmetro 'Ord' dos nós.

```

Transaction #1

em TC (L.EIC010):
Turma 1LEIC01 (Sexta, 10:30) -> (Quinta, 8:30) -> HSM -> B301

Transaction #2

em AC (L.EIC006):
Turma 1LEIC01 (Quinta, 8:30) -> (Sexta, 10:30) -> B336

Transaction #3

em AM II (L.EIC007):
Turma 1LEIC01 (Terça, 8:30) -> (Segunda, 16:00) -> J002

em AC (L.EIC006):
Turma 1LEIC01 (Sexta, 10:30) -> PMSP
  
```

Figura 3.10: Mensagem de exportação

O resultado final da leitura do grafo está apresentado na Figura 3.10. Para gerar este grafo, foi necessário

reunir primeiro o conjunto de alterações realizadas nos horários, através da comparação entre a base de dados inicial e a base de dados final (após a edição dos horários). Este conjunto de alterações é então ordenado por ordem crescente de ano letivo e códigos das unidades curriculares e turmas. Para cada uma das alterações, é criado um nó no grafo. Em seguida, cada nó é visitado e a alteração correspondente é aplicada na base de dados inicial para verificar se causa algum conflito. Caso isso ocorra, é feita uma pesquisa pelos nós não visitados para encontrar aquele que resolve o conflito gerado, aplicando-o na base de dados e verificando se este causa novos conflitos, realizando assim uma pesquisa em profundidade.

Através desta organização das alterações dos horários, obtêm-se conjuntos menores de modificações, tornando a exportação substancialmente mais prática. Apenas é necessário ler o grafo para obter as novas sequências de alterações que garantem a inexistência de conflitos no início e no fim dessas sequências. Desta forma, assegura-se que todos os conflitos gerados por uma alteração são resolvidos por outra subsequente na mesma sequência.

Capítulo 4

Conclusões

Neste capítulo são sumariados os resultados alcançados e as lições aprendidas. Finalmente, são apresentadas as limitações do trabalho e são propostas melhorias e trabalho futuro.

4.1 Resultados alcançados

Um dos principais objectivos do projeto passava por melhorar um sistema pré-existente, e como tal, aumentar a eficiência do *parser* utilizado era crucial. Para tal, e após uma análise do código existente, foi possível identificar operações que eram desnecessárias, o que levou a uma reconstrução de algumas funções através de otimização de código e *refactor*. Também na base de dados foi possível eliminar alguma informação redundante. Estas alterações, em conjunto com as correções na interface e na exportação de alterações de horários, resultaram numa melhoria global de performance que pode ser resumida a:

- Redução do tempo do *parse* em 62%
- Redução da dimensão da base de dados em 40%
- Redução da geração de blocos vermelhos em 99.81%
- Melhoria na apresentação das alterações exportadas

4.2 Lições aprendidas

As lições aprendidas ao longo do desenvolvimento do projeto podem ser aplicadas em projetos futuros para melhorar processos e resultados. Aqui se destacam algumas das mais importantes:

1. **Importância da Comunicação:** A comunicação clara e regular entre os membros da equipa e o tutor foi crucial para o sucesso do projeto. As reuniões diárias e semanais ajudaram a manter todos os membros alinhados e permitiram resolver problemas rapidamente.
2. **Valor das Ferramentas Colaborativas:** A utilização de plataformas como o *Overleaf* para a elaboração do relatório e o *Discord* para comunicação entre a equipa e partilha de ecrã, facilitou a colaboração e a consistência do projeto. Consideramos estas ferramentas essenciais para a eficiência e a qualidade do trabalho.
3. **Gestão de Tempo e Recursos:** A gestão eficiente do tempo e dos recursos disponíveis foi uma lição importante. O planeamento deverá ser sempre cuidadoso e sobretudo realista/adequado às

tarefas propostas com o objetivo de garantir que os prazos sejam cumpridos e que os recursos sejam utilizados da melhor maneira. Aqui, não devemos subestimar determinadas tarefas, pois poderá resultar num mau planeamento e obrigar a mudanças ao longo do projeto.

4. **Adaptação às Mudanças:** Ainda de acordo com o ponto anterior, neste projeto, inicialmente a ideia passava por desenvolver uma nova interface com recurso a outras *frameworks*. No entanto, o tempo para a melhoria do *parser* foi subestimado, o que levou a ter de redefinir objetivos e a manter a interface existente, optando por uma melhoria da mesma.
5. **Importância do Feedback:** O feedback contínuo do tutor foi vital para o desenvolvimento do projeto. Nomeadamente no esclarecimento de dúvidas e estabelecimento de prioridades.

4.3 Sugestões futuras

Apesar das melhorias alcançadas, é reconhecido que há ainda espaço para desenvolvimentos futuros que podem adicionar valor significativo à aplicação. As primeiras duas sugestões eram já objetivos deste trabalho, mas acabaram por não ser implementadas.

1. **Mais formas de visualização:** Adicionar mais layouts de visualização dos respetivos horários, como por exemplo visualizações em lista, calendários mensais, semanais.
2. **Funcionalidade Drag-and-Drop:** A implementação de uma funcionalidade de *drag & drop* no que diz respeito à troca de aulas proporcionará aos utilizadores realizar trocas de forma mais intuitiva e eficiente, resultando numa melhor experiência de utilizador. Atualmente, para efetuar uma troca, é necessário realizar um "click" sobre a aula que se pretende trocar, e outro "click" sobre o bloco onde se pretende concluir a troca.
3. **Criação de dashboards interativos:** A adição de vários dashboards que forneçam uma visão mais abrangente e dinâmica de vários indicadores de desempenho (KPI's), levará a uma melhor visibilidade e controlo sobre algumas importantes métricas, tais como:
 - **Lotação de salas:** Ocupação percentual de cada sala.
 - **Percentagem de conflitos:** Número e percentagem de conflitos nos horários.
 - **Disponibilidade de docentes:** Visão detalhada das disponibilidades, ou falta dela, por parte dos docentes.

Em suma, com estas propostas futuras, espera-se proporcionar não só uma melhoria na experiência de utilizador, mas também uma melhor eficácia na gestão dos horários principalmente aos docentes que utilizam esta plataforma no desempenho das suas funções.

Referências bibliográficas

- [1] Andrea Schaerf. “A survey of automated timetabling”. Em: *Artificial intelligence review* 13 (1999), pp. 87–127.
- [2] Gemechu Taye et al. “Exploring the Role of Artificial Intelligence in Class Scheduling and Management: A Comprehensive Survey and Review”. Em: *2023 International Conference on Computer Science and Emerging Technologies (CSET)*. IEEE. 2023, pp. 1–11.
- [3] Khavee Agustus W Botangen. “Web-Based Class Scheduling for a Collaborative Preparation of Block-Based Schedules”. Em: *International Proceedings of Economics Development and Research* 81 (2014), p. 161.
- [4] Bárbara Carvalho et al. *Análise e Gestão de Horários da L.EIC*. Rel. téc. Faculdade de Engenharia da Universidade do Porto, 2023.
- [5] FullCalendar. *FullCalendar*. <https://fullcalendar.io/>. [Online; acessado a 21 de março de 2024]. 2024.
- [6] Toast UI. *Toast UI Calendar*. <https://ui.toast.com/tui-calendar>. [Online; acessado a 21 de março de 2024]. 2022.
- [7] jqense. *React Big Calendar*. <https://jqense.github.io/react-big-calendar/examples/index.html?path=/story/about-big-calendar--page>. [Online; acessado a 21 de março de 2024]. 2024.
- [8] Sass Team. *Sass*. <https://sass-lang.com/>. [Online; acessado a 15 de junho de 2024]. 2024.